

CENTRO UNIVERSITÁRIO DO ESTADO DO PARÁ - CESUPA

ESCOLA DE NEGÓCIOS, TECNOLOGIA E INOVAÇÃO

DISCIPLINA: TEORIA DA COMPUTABILIDADE

**RELATÓRIO DE FORMALIZAÇÃO, IMPLEMENTAÇÃO E RASTREAMENTO DE
EXECUÇÃO**

**Modelos Computacionais: APD, λ -Cálculo e Máquina de Turing Não
Determinística**

Professor: Daniel Leal Souza

Turma: CC5MA

Integrantes:

- João Pedro Almeida Follmann:
- Yuri Antonio Santos Fernandes:
- Samuel Paula Nunes Salheb:
- Arthur José Aviz Lima:
- Gabriel Cruz Filgueira:

Belém - PA
2026

RESUMO

Este relatório apresenta a formalização, a implementação e o rastreamento de execução de três modelos computacionais estudados na disciplina Teoria da Computabilidade: um Autômato com Pilha Determinístico para reconhecimento de expressões aritméticas, um avaliador de λ -Cálculo com Numerais de Church e uma Máquina de Turing Não Determinística para o problema Subset Sum em representação unária. O objetivo é demonstrar, por meio de modelos distintos, diferentes formas de processamento computacional: memória em pilha, redução funcional e ramificação não determinística. Cada implementação resolve um problema próprio, possui testes documentados e apresenta limitações técnicas associadas ao modelo utilizado.

Palavras-chave: computabilidade; autômato com pilha; λ -cálculo; máquina de Turing não determinística; rastreamento de execução.

SUMÁRIO

- 1 INTRODUÇÃO
- 2 ORGANIZAÇÃO DO PROJETO
- 3 AUTÔMATO COM PILHA DETERMINÍSTICO
- 4 λ -CÁLCULO COM NUMERAIS DE CHURCH
- 5 MÁQUINA DE TURING NÃO DETERMINÍSTICA
- 6 COMPARAÇÃO ENTRE OS MODELOS
- 7 CONSIDERAÇÕES FINAIS
- REFERÊNCIAS
- DECLARAÇÃO DE USO DE IA

1 INTRODUÇÃO

A atividade desenvolvida tem como finalidade relacionar teoria formal e implementação prática de modelos computacionais. Foram escolhidos três modelos distintos: Autômato com Pilha Determinístico, λ -Cálculo e Máquina de Turing Não Determinística. A escolha foi feita para cobrir mecanismos computacionais diferentes e permitir comparação entre reconhecimento de linguagens, avaliação funcional e exploração de múltiplos ramos de computação.

O primeiro modelo é um Autômato com Pilha Determinístico implementado no JFLAP, utilizado para reconhecer expressões aritméticas sintaticamente válidas. O segundo modelo é um avaliador de λ -Cálculo implementado em Python, com Numerais de Church e operações de sucessor, soma e multiplicação. O terceiro modelo é uma Máquina de Turing Não Determinística simulada por programa, voltada ao reconhecimento de instâncias do problema Subset Sum em unário.

O relatório registra a estrutura formal de cada modelo, o problema resolvido, a estratégia de implementação, os testes realizados, os rastreamentos de execução e as limitações observadas.

2 ORGANIZAÇÃO DO PROJETO

O projeto foi organizado em repositório GitHub, com separação entre implementações, testes, rastreamentos, referências, slides e relatório. A estrutura busca facilitar a reprodução dos experimentos e a verificação dos resultados.

Modelo	Problema resolvido	Ferramenta
APD	Reconhecimento de expressões aritméticas sintaticamente válidas	JFLAP (.jff)
λ -Cálculo	Calculadora funcional com Numerais de Church	Python / Jupyter Notebook
MT Não Determinística	Reconhecimento de Subset Sum em unário	Python

A organização separa os arquivos por finalidade: implementações em `implementacoes/`, casos de teste em `testes/`, rastreamentos em `rastreamentos/`, referências em `referencias.md` e declaração de uso de IA em `uso_ia.md`.

3 AUTÔMATO COM PILHA DETERMINÍSTICO

3.1 Contextualização

O Autômato com Pilha é um modelo formado por um controle finito e uma pilha. Ele reconhece linguagens livres de contexto e é mais expressivo que um autômato finito, pois consegue armazenar informação de forma hierárquica. Neste projeto, o uso da pilha é aplicado ao controle de abertura e fechamento de parênteses em expressões aritméticas.

3.2 Problema resolvido

A implementação reconhece expressões aritméticas formadas por variáveis, operadores e parênteses. A máquina verifica se a sequência de símbolos respeita a estrutura sintática esperada e se os parênteses são abertos e fechados corretamente. A implementação não calcula o valor da expressão, nem aplica regras de precedência entre operadores.

Alfabeto de entrada: $\Sigma = \{a, b, c, +, -, *, /, (,)\}$.

Exemplos de cadeias aceitas: $a+b$, $a-b$, $(a+b)$, $((c-b))$ e $((a-b)*c)$.

Exemplos de cadeias rejeitadas: a^* , $()$, $a+b)$, $(b+$ e $((a)))$.

3.3 Formalização

O APD pode ser descrito pela tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$, em que:

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9\}$;
- $\Sigma = \{a, b, c, +, -, *, /, (,)\}$;
- $\Gamma = \{Z, P\}$;
- q_0 é o estado inicial;
- Z é o símbolo inicial da pilha;
- $F = \{q_9\}$;
- δ é a função de transição definida no arquivo JFLAP.

O critério de aceitação é por estado final, após o consumo completo da entrada. O símbolo P é utilizado para marcar abertura de parêntese, e o símbolo Z representa a base da pilha.

3.4 Implementação

A implementação foi construída no JFLAP no formato .jff, com dez estados. O arquivo principal é `pda_expressoes_aritmeticas_corrigido.jff`, localizado na pasta `implementacoes/apd_expressoes/`.

A dinâmica principal é: ao ler uma variável, a máquina avança para um estado que espera operador ou finalização; ao ler operador, a máquina passa a esperar uma nova variável ou subexpressão; ao ler abertura de parêntese, empilha P; ao ler fechamento de parêntese, desempilha P.

3.5 Testes

Entrada	Resultado esperado	Justificativa
a+b	aceita	expressão simples variável-operador-variável
a-b	aceita	expressão simples com subtração
a*	rejeita	operador sem operando à direita
(a+b*c)	aceita	parênteses com expressão sintaticamente válida
((c-b))	aceita	dois níveis de aninhamento
()	rejeita	parênteses vazios
a+b)	rejeita	fechamento sem abertura correspondente
(b+)	rejeita	operador sem operando antes do fechamento
((a-b)*c)	aceita	dois níveis e operador externo
((a)))	rejeita	ultrapassa o limite de aninhamento adotado

3.6 Rastreamento de execução

Entrada analisada: (a+b).

Passo	Estado	Entrada restante	Pilha	Ação
1	q0	(a+b)	[Z]	lê (e empilha P
2	q3	a+b)	[P,Z]	lê variável
3	q4	+b)	[P,Z]	lê operador
4	q5	b)	[P,Z]	lê variável
5	q4	)	[P,Z]	lê) e desempilha P
6	q1	ε	[Z]	transição vazia para q9
7	q9	ε	[Z]	aceita por estado final

3.7 Limitações

- A máquina realiza validação sintática, mas não avalia a expressão numericamente.
- Não há diferenciação de precedência entre operadores.
- A implementação adota limite prático de aninhamento conforme a construção feita no JFLAP.
- O alfabeto é restrito às variáveis a, b e c e aos operadores definidos.

4 λ-CÁLCULO COM NUMERAIS DE CHURCH

4.1 Contextualização

O λ-Cálculo é um modelo formal de computação baseado em abstração, aplicação e substituição. Diferentemente de máquinas baseadas em estados, a computação ocorre pela redução de expressões funcionais. O modelo é equivalente em poder computacional à Máquina de Turing e serve como fundamento teórico para o paradigma funcional.

4.2 Problema resolvido

A implementação desenvolve uma calculadora funcional com Numerais de Church. Números naturais são representados como funções de alta ordem: o número n é modelado como uma função que aplica f exatamente n vezes sobre x . Foram implementadas as operações SUCC, ADD e MULT.

Conceito	Definição
ZERO	$\lambda f. \lambda x. x$
UM	$\lambda f. \lambda x. f\ x$
DOIS	$\lambda f. \lambda x. f\ (f\ x)$
SUCC	$\lambda n. \lambda f. \lambda x. f\ (n\ f\ x)$
ADD	$\lambda m. \lambda n. \lambda f. \lambda x. m\ f\ (n\ f\ x)$
MULT	$\lambda m. \lambda n. \lambda f. m\ (n\ f)$

4.3 Formalização

A sintaxe dos termos λ utilizada no projeto é dada por: $\Lambda ::= x \mid (\lambda x. \Lambda) \mid (\Lambda\ \Lambda)$.

- Variável: representa uma referência simples dentro de um termo.
- Abstração: representa uma função anônima, como $\lambda x. E$.
- Aplicação: representa a aplicação de um termo a outro, como $E_1\ E_2$.
- Redução β : substituição de uma variável ligada pelo argumento aplicado.
- Conversão α : renomeação de variável ligada para evitar captura de variáveis livres.

A estratégia adotada é a redução em ordem normal, reduzindo primeiro o redex mais externo e mais à esquerda. O critério de parada ocorre quando não existem mais redexes redutíveis.

4.4 Implementação

A implementação foi feita em Python, com representação explícita da árvore sintática abstrata. As classes Var, Abs e App representam variáveis, abstrações e

aplicações. A função de substituição verifica variáveis livres e realiza conversão α quando há risco de captura.

Arquivos principais: `lambda_church_numerais.py` e `lambda_church_numerais.ipynb`, localizados em `implementacoes/lambda_church/`.

4.5 Testes

Caso	Expressão	Tipo	Passos	Resultado
1	SUCC ZERO	padrão	3	1
2	SUCC UM	padrão	3	2
3	ADD UM DOIS	padrão	7	3
4	ADD DOIS DOIS	padrão	9	4
5	MULT DOIS TRES	complexo	12	6
6	SUCC (ADD UM UM)	composto	10	3
7	ADD (SUCC DOIS) DOIS	composto	12	5
8	MULT TRES ZERO	fronteira	7	0
9	ADD ZERO DOIS	fronteira	5	2
10	Var("x")	termo livre	0	estável; não representa numeral de Church

4.6 Rastreamento de execução

Exemplo analisado: ADD UM DOIS. A expressão representa a soma $1 + 2$. Após sucessivas reduções β , a forma normal obtida corresponde ao numeral de Church 3.

Etapa	Descrição
1	Aplicação de ADD aos numerais UM e DOIS.
2	Redução β substitui o parâmetro m por UM.
3	Redução β substitui o parâmetro n por DOIS.
4	Aplicação das funções internas f e x.
5	Composição das aplicações de f produz três ocorrências sucessivas.
6	Forma normal: $\lambda f. \lambda x. f(f(f\ x))$.
7	Decodificação estrutural: resultado 3.

4.7 Limitações

- O tamanho textual dos termos intermediários cresce rapidamente.
- O λ -Cálculo não tipado admite termos que não terminam.
- A decodificação para inteiro é feita apenas para formas compatíveis com Numerais de Church.
- A execução é adequada para demonstração teórica, não para eficiência prática.

5 MÁQUINA DE TURING NÃO DETERMINÍSTICA

5.1 Contextualização

A Máquina de Turing Não Determinística é uma modificação da Máquina de Turing na qual uma configuração pode possuir mais de uma transição possível. A computação deixa de ser uma sequência única e passa a formar uma árvore de configurações. A entrada é aceita se pelo menos um ramo da árvore atinge um estado de aceitação.

5.2 Problema resolvido

A implementação reconhece instâncias do problema Subset Sum em unário. A entrada contém uma lista de números representados por blocos de letras a e um alvo também em unário. A máquina aceita quando existe algum subconjunto dos números cuja soma seja exatamente igual ao alvo.

Formato da entrada: bloco1#bloco2#...#blocoN#alvo.

Exemplo: aa#aaa#a=aaaa. Interpretação: {2, 3, 1}, alvo 4. A entrada é aceita porque $3 + 1 = 4$.

5.3 Formalização

A simulação usa configurações da forma (estado, índice, soma, números, alvo, caminho). O estado indica a etapa da computação, o índice indica o próximo número a ser decidido, a soma registra o valor acumulado, e o caminho registra as escolhas feitas no ramo.

Estado	Função
q0_validar_entrada	validar a sintaxe da entrada
q1_escolher_numero	decidir o próximo número
q2_incluir_numero	criar ramo que inclui o número atual
q3_ignorar_numero	criar ramo que ignora o número atual
q4_comparar_soma	comparar soma acumulada com o alvo
qacc_aceita	estado final de aceitação
qrej_rejeita	estado final de rejeição do ramo

A relação de transição não determinística central ocorre em q1. Para cada número ainda não decidido, a máquina cria dois ramos: um que inclui o número na soma e outro que o ignora.

Origem	Condição	Destino	Ação
q0	entrada válida	q1	inicializa soma e índice

q1	há número a decidir	q2	ramo inclui número atual
q1	há número a decidir	q3	ramo ignora número atual
q1	todos decididos	q4	compara soma com alvo
q2	número atual = n	q1	soma n e avança
q3	número atual = n	q1	mantém soma e avança
q4	soma = alvo	qacc	aceita
q4	soma $\neq$ alvo	qrej	rejeita o ramo

5.4 Implementação

A implementação foi feita em Python, com classes para configuração instantânea, nó da árvore de computação e resultado da execução. O objetivo do programa não é usar uma função pronta de Subset Sum, mas representar explicitamente os estados, as configurações, a relação de transição e os ramos de computação.

Arquivo principal: `mtnd_subset_sum.py`, localizado em `implementacoes/mt_nao_deterministica_subset_sum/`.

Comandos principais:

```
python
implementacoes/mt_nao_deterministica_subset_sum/mtnd_subset_sum.py
"aa#aaa#a=aaaa" --tree
python
implementacoes/mt_nao_deterministica_subset_sum/mtnd_subset_sum.py --
testes
python
implementacoes/mt_nao_deterministica_subset_sum/mtnd_subset_sum.py --
transicoes
```

5.5 Testes

Entrada	Interpretação	Esperado	Obtido	Caminho aceitante
aa#aaa#a=aaaa	{2,3,1}, alvo 4	aceita	aceita	-2 +3 +1
a#aa#aaa=aaaaaa	{1,2,3}, alvo 6	aceita	aceita	+1 +2 +3
aa#aaaa=aaa	{2,4}, alvo 3	rejeita	rejeita	
aaa#aa=aaaa	{3,2}, alvo 4	rejeita	rejeita	
a#aa#aaa=	alvo vazio	rejeita	rejeita	
aa##aaa=aaaa	bloco vazio	rejeita	rejeita	

5.6 Rastreamento de execução

Entrada analisada: aa#aaa#a=aaaa. Números: [2, 3, 1]. Alvo: 4. O ramo aceitante encontrado ignora o número 2, inclui o número 3 e inclui o número 1.

```
soma 0
- inclui 2 -> soma 2
- inclui 3 -> soma 5
  - inclui 1 -> soma 6 -> rejeita
  - ignora 1 -> soma 5 -> rejeita
- ignora 3 -> soma 2
  - inclui 1 -> soma 3 -> rejeita
  - ignora 1 -> soma 2 -> rejeita
- ignora 2 -> soma 0
  - inclui 3 -> soma 3
    - inclui 1 -> soma 4 -> aceita
    - ignora 1 -> soma 3 -> rejeita
  - ignora 3 -> soma 0
    - inclui 1 -> soma 1 -> rejeita
    - ignora 1 -> soma 0 -> rejeita
```

5.7 Limitações

- A árvore de computação cresce exponencialmente em relação ao número de blocos de entrada.
- A representação é unária, adequada para fins didáticos, mas pouco eficiente para valores grandes.
- O simulador possui limite de configurações para evitar execuções excessivamente longas.
- A implementação representa uma MTND de forma simulada, não como arquivo JFLAP.

6 COMPARAÇÃO ENTRE OS MODELOS

Os três modelos escolhidos demonstram formas diferentes de computação. O APD utiliza pilha para lidar com estruturas aninhadas; o λ -Cálculo computa por substituição funcional; e a MT Não Determinística explora escolhas alternativas por ramificação.

Modelo	Mecanismo principal	O que demonstra	Limitação principal
APD	Controle finito + pilha	Reconhecimento sintático com memória LIFO	Não avalia expressões e tem alfabeto restrito
λ -Cálculo	Redução β e substituição	Computação por funções puras	Pode não terminar e cresce em espaço
MTND	Árvore de configurações	Aceitação por existência de ramo aceitante	Crescimento exponencial de ramos

Em termos de computabilidade, o λ -Cálculo e a Máquina de Turing são modelos universais equivalentes em poder de expressão. O APD é menos poderoso, pois se restringe ao reconhecimento de linguagens livres de contexto. A comparação ajuda a evidenciar a diferença entre reconhecer padrões sintáticos, executar reduções funcionais e modelar escolhas não determinísticas.

7 CONSIDERAÇÕES FINAIS

O projeto reuniu três implementações distintas e complementares. A APD mostrou o uso de pilha na validação de expressões aritméticas, o λ -Cálculo mostrou a execução de operações aritméticas sem primitivas numéricas, e a MT Não Determinística mostrou a construção de uma árvore de computação para o problema Subset Sum.

Os testes e rastreamentos indicam que as implementações seguem os problemas definidos e permitem reprodução dos resultados. As limitações registradas também são relevantes para a análise técnica, pois mostram diferenças entre modelo teórico, implementação didática e custo computacional prático.

REFERÊNCIAS

CHURCH, Alonzo. An Unsolvable Problem of Elementary Number Theory. American Journal of Mathematics, v. 58, n. 2, 1936, p. 345-363.

DIVERIO, Tiarajú Asmuz; MENEZES, Paulo Blauth. Teoria da Computação: Máquinas Universais e Computabilidade. 3. ed. Porto Alegre: Bookman, 2011.

JFLAP. JFLAP: An Interactive Formal Languages and Automata Package. Material de apoio e documentação da ferramenta.

MENEZES, Paulo Blauth. Linguagens Formais e Autômatos. 6. ed. Porto Alegre: Bookman, 2011.

TURING, Alan M. On Computable Numbers, with an Application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, 1936.

Slides da disciplina de Teoria da Computabilidade - Máquinas de Turing, modificações da Máquina de Turing, Autômatos de Pilha, λ -Cálculo e computabilidade.

DECLARAÇÃO DE USO DE IA

Ferramentas utilizadas

Foram utilizadas as ferramentas ChatGPT 5.5 e Claude, nas datas 03/06/2026 e 09/06/2026 respectivamente

Finalidade do uso

- Apoio à organização das ideias de implementação;
- Revisão da estrutura do repositório;
- Sugestão de testes e rastreamentos;
- Apoio na modelagem da Máquina de Turing Não Determinística para Subset Sum em unário;
- Apoio na organização inicial do relatório e dos materiais auxiliares, como os slides.

Resumo dos prompts utilizados

Foram solicitadas sugestões de máquinas/modelos adequados à proposta da atividade, revisão da documentação da APD, análise da máquina em λ -Cálculo, criação de uma estrutura de repositório e apoio na elaboração de uma Máquina de Turing Não Determinística com árvore de computação explícita. Também foi solicitado apoio para a criação e estruturação do relatório e dos slides utilizados na apresentação

Trechos aproveitados

Foram aproveitados rascunhos de estrutura, sugestões de teste, textos explicativos iniciais, orientação de organização do GitHub e uma versão base do simulador da MT Não Determinística.

Modificações, correções e validação da equipe

Foi modificada a primeira versão feita da máquina APD que teve apoio da ferramenta Claude, com o intuito de melhorar a apresentação dela. As primeiras versões da documentação do GitHub também foram posteriormente revisadas e corrigidas pela equipe. Por fim, o slide teve novas adições para enriquecer a apresentação.

Declaração

Declaramos que o uso de IA teve caráter de apoio e não substituiu o estudo, a validação técnica, a implementação e a compreensão dos modelos computacionais utilizados. Declaramos também que todos os integrantes revisaram os materiais incorporados e são capazes de explicar os conceitos, arquivos, testes, rastreamentos e decisões técnicas presentes no projeto.